SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE:CSA5113
COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
CASE STUDY

Code of Conduct:

I DINDU PUJITHA(192373037) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *D PUJITHA*

Annexure A
Case Study

1. **A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.**

MD5 Password Hashing and Database Breach

Scenario:

A software company developed an online employee management system where users create accounts using usernames and passwords. To protect passwords, the development team stored the passwords in the database after applying the MD5 hashing algorithm.

After several years, the company's database was attacked and stolen. The attackers obtained the MD5 password hashes. Using password-cracking tools and precomputed password databases, they were able to recover the passwords of many employees.

Problem:

The company assumed that converting passwords into MD5 hashes would make them secure. However, MD5 is an outdated hashing algorithm and is not suitable for password storage.

Analysis:

MD5 produces a 128-bit hash and is designed to be computationally fast. This is a major disadvantage for password protection. Attackers can calculate a very large number of MD5 hashes in a short period using modern GPUs and specialized cracking tools.

Another weakness is that MD5 does not automatically use a **salt**. If two users choose the same password, their hashes will be identical. Attackers can also use rainbow tables and dictionary attacks to identify commonly used passwords.

MD5 has additionally been demonstrated to have serious cryptographic collision weaknesses. Although collision resistance is not the only issue involved in password storage, it is another reason why MD5 should not be used in modern security systems.

Proposed Solution:

The company should migrate from MD5 to a modern password-hashing method. If a SHA-based solution is specifically required, PBKDF2-HMAC-SHA-256 or PBKDF2-HMAC-SHA-512 can be used with a unique random salt and an appropriate work factor.

For new applications, memory-hard password hashing algorithms such as Argon2id are generally preferred.

The improved process is:

Password → Random Salt + Password → Password-Hashing Algorithm → Stored Hash

The salt should be unique for every user and stored alongside the resulting hash.

Conclusion:

The security breach occurred partly because MD5 is too fast and outdated for password storage. Replacing MD5 with a salted, computationally expensive password-hashing scheme will make password cracking significantly more difficult.

2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.

Unauthorized Banking Transactions Through Digital Signature

Scenario:

A bank introduced a digital-signature system for approving online money transfers. Whenever a customer initiated a transaction, the system generated a digital signature to confirm that the transaction had been authorized.

After deployment, the bank noticed several unauthorized transactions that had been successfully approved. Investigation showed that some attackers were able to submit transactions that passed the application's signature verification process.

Problem:

The bank's digital-signature implementation was not correctly protecting the transaction information. In particular, the signature process did not sufficiently bind all important transaction details to the signed data.

Analysis:

A digital signature provides two important security properties:

Authentication – verifies the identity of the signer.

Integrity – verifies that the signed information has not been changed.

If the application signs only a transaction identifier instead of critical information such as the amount and destination account, an attacker may be able to manipulate transaction details without invalidating the signature.

Another possible weakness is poor protection of the user's private signing key. If an attacker obtains the private key, they can create valid signatures.

The bank may also be using an outdated or improperly configured signature algorithm or may not be correctly checking certificate validity.

Proposed Solution:

The bank should use modern and properly implemented digital-signature standards such as:

- **RSA-PSS**
- **ECDSA**
- **EdDSA**, where supported and appropriate

The signature should cover all critical transaction information:

Transaction ID + Sender + Receiver + Amount + Timestamp/Nonce → Digital Signature

The bank should also:

- Protect private keys using secure key storage or HSMs.
- Properly verify every digital signature.
- Validate certificates and their trust chains where certificates are used.
- Implement certificate revocation checking where appropriate.
- Use strong customer authentication for sensitive transactions.
- Maintain transaction and security audit logs.
- Prevent reuse of old transaction authorization data.

Conclusion:

The unauthorized transactions were possible because the digital-signature implementation did not sufficiently protect the complete transaction and/or signing keys. Proper transaction binding,

secure key management, strong signature algorithms, and complete signature verification are required.

3. **An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.**

Kerberos Replay Attack in an Enterprise Network

Scenario:

A large organization uses **Kerberos** to authenticate employees accessing internal servers. Employees log in once and receive Kerberos tickets that allow them to access authorized services.

An attacker monitoring network traffic captures authentication-related information from a legitimate user's session. Later, the attacker attempts to reuse the captured information to access an internal service.

Problem:

The attacker is attempting a **replay attack**, where a previously valid authentication message is transmitted again to gain unauthorized access.

Analysis:

Kerberos is designed to protect against replay attacks using mechanisms such as **timestamps and authenticators**. An authenticator contains information that helps demonstrate that the request is recent.

However, replay attacks can become possible or more likely if the Kerberos implementation is incorrectly configured. For example:

An attacker who captures a valid authentication message may attempt to resend it before the server considers it invalid.

Proposed Solution:

The organization should strengthen its Kerberos configuration by:

1. Using timestamps and authenticators correctly.
2. Maintaining accurate time synchronization between clients and servers.

3. Enforcing an appropriate clock-skew limit.
4. Using replay caches to identify previously used authenticators.
5. Limiting ticket lifetimes where appropriate.
6. Protecting Kerberos tickets and session keys.
7. Monitoring authentication logs for suspicious repeated requests.
8. Keeping the Kerberos/KDC infrastructure properly secured and updated.

The authentication process should ensure that an old request cannot simply be reused.

Conclusion:

The attack occurred because a valid authentication message was captured and reused. Correct timestamp validation, replay-cache mechanisms, secure ticket management, and synchronized clocks can prevent or significantly reduce Kerberos replay attacks.

4. **A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.**

Message Modification and HMAC

Scenario:

A financial company developed an internal communication system to exchange transaction messages between two branches. To verify message integrity, the developers calculated a normal SHA-256 hash of every message.

For example:

Message:

Transfer ₹50,000 to Account A

SHA-256Hash:

Hash Value

The receiving branch recalculated the hash and compared it with the received value.

An attacker intercepted the communication and changed the message to:

ModifiedMessage:

Transfer ₹5,00,000 to Account B

The attacker also calculated a new SHA-256 hash and replaced the original hash with the new one.

The receiving system accepted the modified message.

Problem:

The system used an ordinary hash function without a secret key. Therefore, the hash provided integrity checking but did not provide authentication against an active attacker.

Analysis:

A normal hash can tell whether the message corresponds to a particular hash value, but anyone who can modify the message can also calculate a new hash.

Therefore:

Message → SHA-256 → Hash

does not prove that the hash was generated by an authorized sender.

Proposed Solution:

The company should use **HMAC**, such as **HMAC-SHA-256**.

HMAC uses a secret key shared between the sender and receiver.

Sender:

Message + Secret Key → HMAC-SHA-256 → Authentication Tag

Receiver:

Received Message + Secret Key → HMAC-SHA-256 → Calculated Tag

The receiver compares the calculated tag with the received tag. If they are different, the message is rejected.

An attacker may be able to modify the message, but without knowing the secret HMAC key, they cannot create a valid authentication tag for the modified message.

Advantages:

HMAC provides:

Message integrity ,Message authentication ,Protection against unauthorized message modification ,Resistance to attackers who do not possess the secret key

Conclusion:

The company's security mechanism failed because it used an unkeyed hash. Replacing the ordinary hash with **HMAC-SHA-256** provides both message integrity and authentication and prevents attackers from simply recalculating the verification value.

5. **An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.**

X.509 Certificate Trust Validation Problems

Scenario:

An organization operates an online employee portal using HTTPS and **X.509 certificates**.

Employees frequently receive browser warnings such as:

"Certificate is not trusted"

or

"Certificate validation failed"

Some employees are unable to establish secure connections to the portal.

Problem:

The organization has problems with its certificate management and trust-chain configuration.

Analysis:

An X.509 certificate is normally issued by a Certificate Authority (CA). The client verifies the certificate and establishes a chain of trust.

The normal structure is:

Server Certificate → Intermediate CA → Root CA

Several problems can cause trust-validation errors.

1. Expired Certificate

The server certificate may have passed its expiration date.

2. Missing Intermediate Certificate

The server may not provide the required intermediate CA certificate. As a result, the client cannot construct the complete trust chain.

3. Untrusted Root CA

The client may not have the appropriate root CA in its trusted certificate store.

4. Hostname Mismatch

The certificate may be issued for one domain while users access another domain.

5. Incorrect System Time

If the client's date or time is incorrect, a valid certificate may appear to be expired or not yet valid.

6. Revoked Certificate

A certificate may have been revoked because its private key was compromised or for another security reason.

7. Weak or Unsupported Configuration

Older cryptographic algorithms or incorrect TLS/certificate configuration may cause compatibility and security problems.

Proposed Solution:

The organization should:

Obtain certificates from trusted Cas,Install the complete certificate chain,Ensure certificates have correct domain names.

Conclusion:

The certificate errors are most likely caused by incorrect certificate configuration, an incomplete trust chain, expired certificates, hostname mismatches, or problems with the trusted CA store. Proper X.509 certificate lifecycle management and trust-chain validation will provide reliable authentication and secure communication.